

The Binary Quadratic Meta-Mask Lexer

A First-Principles Method for Validating Fixed-Width Instruction Words

Abstract

This paper introduces a method for validating fixed-width instruction words without treating lexing as a sequence of conditional parsing steps.

The central idea is simple:

A valid instruction word is a point on an algebraic surface.

Instead of reading a token from left to right and branching at each delimiter, the instruction is divided into fixed-size segments. The required constants, repeated variables, and free payload fields are then encoded into a single quadratic error function.

The instruction is accepted if and only if the quadratic evaluates to zero.

$Q(S) = 0 \rightarrow \text{valid instruction}$
 $Q(S) > 0 \rightarrow \text{malformed instruction}$

This creates a lexer that behaves like a mathematical proof rather than a parser. The frame is validated as a whole. Payload variables are left untouched. A runtime can then use those payload variables as coordinates, ray paths, octree selectors, bitboard indices, or memory displacement fields.

The result is a branch-minimized instruction format where structure, validation, and geometric traversal are unified.

1. The Problem

Most lexers are sequential.

They read a stream, check the first symbol, branch, check the next symbol, branch again, and continue until the token is accepted or rejected.

That model works, but it has several costs:

```
branching
state machines
delimiter scanning
partial parses
error recovery paths
string-level interpretation
```

For high-throughput systems, especially systems that operate on fixed-width binary or hexadecimal tokens, this is unnecessary.

If the instruction has a known shape, the lexer does not need to discover that shape one character at a time. It can evaluate the whole word at once.

The question becomes:

```
Can instruction validity be expressed as one arithmetic invariant?
```

The answer is yes.

2. The Core Idea

Take a fixed-width instruction word and split it into eight 16-bit segments:

```
S0 S1 S2 S3 S4 S5 S6 S7
```

Each segment is a 16-bit integer.

Some segments are fixed constants.

Some contain repeated variables.

Some contain free payload data.

A token is valid when:

1. every fixed constant matches exactly;
2. every repeated variable agrees with its other appearances;
3. payload fields are allowed to vary freely.

This can be expressed as a quadratic error function.

Every violation contributes a positive square.

Every valid field contributes zero.

Because squares cannot cancel each other, the total is zero if and only if all constraints are satisfied.

3. The Instruction Layout

Define the instruction shape:

```
[0xLL00]-0x03bf-0xNNNN-0x2bLL-0x2fLL-0xMMMM-0x039f-[0xLLff]
```

This is an eight-segment word:

```
S0 = 0xLL00  
S1 = 0x03bf  
S2 = 0xNNNN  
S3 = 0x2bLL  
S4 = 0x2fLL  
S5 = 0xMMMM  
S6 = 0x039f  
S7 = 0xLLff
```

The variables are:

```
LL   = local layer / lens / lane variable  
NNNN = free payload variable N  
MMMM = free payload variable M
```

The fixed markers are:

```
0x03bf = chiral delimiter  
0x039f = cardinal delimiter  
0x2b   = positive branch marker  
0x2f   = projection branch marker  
0x00   = lower boundary  
0xff   = upper closure boundary
```

The instruction contains both structure and payload.

The structure is carried by:

```
S0, S1, S3, S4, S6, S7
```

The payload is carried by:

S2, S5

The lexer validates the structure while leaving the payload untouched.

4. Extracting the Repeated Variable

The variable `LL` appears four times:

```
S0 high byte  
S3 low byte  
S4 low byte  
S7 high byte
```

Define:

```
L0 = high8(S0)  
L3 = low8(S3)  
L4 = low8(S4)  
L7 = high8(S7)
```

A valid instruction requires:

```
L0 = L3 = L4 = L7
```

Instead of testing all pairs, it is enough to test a chain:

```
L0 = L3  
L3 = L4  
L4 = L7
```

If the chain holds, all four are equal.

The variable-coherence error is:

```
Evar =  
  (L0 - L3)^2  
+ (L3 - L4)^2  
+ (L4 - L7)^2
```

This equals zero if and only if all `LL` instances agree.

5. Validating the Fixed Constants

The fixed parts of the instruction are:

```
low8(S0)  = 0x00
S1        = 0x03bf
high8(S3) = 0x2b
high8(S4) = 0x2f
S6        = 0x039f
low8(S7)  = 0xff
```

The constant-alignment error is:

```
Econst =
  low8(S0)^2
+ (S1 - 0x03bf)^2
+ (high8(S3) - 0x2b)^2
+ (high8(S4) - 0x2f)^2
+ (S6 - 0x039f)^2
+ (low8(S7) - 0xff)^2
```

This equals zero if and only if every constant field is correct.

6. The Binary Quadratic Meta-Mask

The full lexer is the sum of the variable-coherence error and the constant-alignment error:

```
Q(S) = Evar + Econst
```

The instruction is valid exactly when:

```
Q(S) = 0
```

The instruction is invalid when:

$$Q(S) > 0$$

This is the complete lexer.

It does not need to ask:

Is this the first delimiter?
Is this the second delimiter?
Which branch am I in?
Where does the payload begin?
Where does the payload end?

Instead it asks one question:

Does this 128-bit word lie on the zero-set of the instruction form?

If yes, the word is valid.

If no, the word is malformed.

7. Why Squares Matter

The use of squares is important.

Without squares, one mismatch could cancel another:

$$+3 + (-3) = 0$$

That would allow malformed instructions to appear valid.

With squares, every mismatch becomes positive:

$$(+3)^2 + (-3)^2 = 9 + 9 = 18$$

No error can cancel another error.

Therefore:

$$Q(S) = 0$$

means every individual constraint is zero.

This gives the lexer a proof-like property:

The whole word is valid if and only if every structural constraint is valid.

8. Payload Pass-Through

The fields `S2` and `S5` do not appear in the quadratic.

```
S2 = 0xNNNN  
S5 = 0xMMMM
```

They are free variables.

This means they can carry payload without affecting frame validity.

Examples of possible interpretations:

```
octree path  
ray coordinate  
bitboard mask  
memory displacement  
child index  
edge selector  
address offset  
symbolic payload
```

The lexer proves the frame.

The payload then passes through the proven frame.

This separation is the key architectural advantage.

9. Octree Raycasting Interpretation

Once the instruction is validated, the free variables can be interpreted geometrically.

For example:

```
N = S2  
M = S5
```

Together, they may form:

```
(N, M)
```

or a concatenated path:

```
N || M
```

In an octree, each group of bits can select a child branch.

Because the frame has already been validated by `Q(S)=0`, the runtime can treat the free variables as traversal data without reparsing the instruction.

The result is:

```
validated frame → safe coordinate interpretation → octree traversal
```

This is why the method is not merely a lexer.

It is a geometric gate.

10. Example: Valid Instruction

Consider:

```
omi-0500-03bf-000c-2b05-2f05-0002-039f-05ff/48
```

The eight segments are:

```
S0 = 0x0500  
S1 = 0x03bf  
S2 = 0x000c  
S3 = 0x2b05  
S4 = 0x2f05
```



```
S5 = 0x0002  
S6 = 0x039f  
S7 = 0x05ff
```

Extract the repeated variable:

```
L0 = 0x05  
L3 = 0x05  
L4 = 0x05  
L7 = 0x05
```

So:

```
Evar = 0
```

Check constants:

```
low8(S0) = 0x00  
S1        = 0x03bf  
high8(S3) = 0x2b  
high8(S4) = 0x2f  
S6        = 0x039f  
low8(S7)  = 0xff
```

So:

```
Econst = 0
```

Therefore:

```
Q(S) = 0
```

The instruction is valid.

The payload is:

```
N = 0x000c  
M = 0x0002
```

Those values may now be interpreted as traversal or execution data.

11. Example: Variable Coherence Failure

Consider:

```
omi-0500-03bf-000c-2b05-2f06-0002-039f-05ff/48
```

Here:

```
L0 = 0x05  
L3 = 0x05  
L4 = 0x06  
L7 = 0x05
```

The coherence error is:

```
Evar =  
  (5 - 5)^2  
+ (5 - 6)^2  
+ (6 - 5)^2  
  
Evar = 0 + 1 + 1 = 2
```

Even if every fixed delimiter is correct:

```
Econst = 0
```

the full form is:

```
Q(S) = 2
```

The instruction is invalid.

12. Example: Delimiter Failure

Consider:

```
omi-0500-03bf-000c-2b05-2f05-0002-039e-05ff/48
```

Here `S6` is wrong:

```
S6 = 0x039e
```

It should be:

```
S6 = 0x039f
```

The delimiter error is:

```
(0x039e - 0x039f)^2 = 1
```

So:

```
Q(S) = 1
```

The instruction is invalid.

13. Runtime Form

In direct implementation, the lexer becomes:

```
extract bytes  
compute Evar  
compute Econst  
return Q(S) == 0
```

The parser is not a state machine.

The parser is a zero-set evaluator.

This gives the central law:

```
Lexing is algebraic membership.
```

A token is not “parsed into validity.”

A token is tested for membership in the valid instruction surface.

14. Practical Result

This formulation has already been validated by tests covering:

```
valid canonical token
valid token with different LL
LL coherence failures
fixed delimiter corruption
free payload passthrough
LL minimum value 0x00
LL maximum value 0xff
address parsing
address round-trip
layout definition
null and malformed rejection
```

The important result is not merely that one example passes.

The important result is that the instruction form has become mechanically testable as a single invariant.

15. Summary

The Binary Quadratic Meta-Mask Lexer converts instruction validation into algebraic surface membership.

```
The instruction is the word.
The mask is the equation.
The lexer is the zero-set.
The payload is the ray.
```

The fixed fields define the frame.

The repeated variable proves local coherence.

The free variables carry traversal data.

The quadratic form proves the whole structure at once.

In one sentence:

A valid instruction is a 128-bit word whose structural error polynomial evaluates to zero.

This is the core principle.

It turns lexing from sequential parsing into direct mathematical validation.